

Lab: Power Calculation, Normal vs Recursive

1. Objective

To calculate x^n using a normal loop method and a recursive fast power method (divide and conquer), and to compare their execution time.

2. How It Works

- **Normal method:** multiply x by itself n times using a loop. For 2^{10} this takes 10 multiplications.
- **Recursive method:** square the base and halve the power at every step. If n is even, $x^n = (x^2)^{n/2}$. If n is odd, $x^n = x \times (x^2)^{(n-1)/2}$. Stop when $n = 0$ and return 1.

Example, 2^{10} : $\text{power}(2,10) \rightarrow \text{power}(4,5) \rightarrow 4 \times \text{power}(16,2) \rightarrow \text{power}(256,1) \rightarrow 256 \times \text{power}(65536,0) = 256$, so the answer is $4 \times 256 = 1024$. Only about 4 steps instead of 10.

3. Algorithm

Normal

1. Set result = 1.
2. Repeat n times: result = result $\times$ x .
3. Return result.

Recursive

1. If $n = 0$, return 1.
2. If n is even, return $\text{power}(x \times x, n / 2)$.
3. If n is odd, return $x \times \text{power}(x \times x, (n - 1) / 2)$.

4. C Code

```
#include <stdio.h>
#include <time.h>

//Normal method
int normalPower(int x, int n)
{
    int result = 1;

    for (int i = 0; i < n; i++)
    {
        result = result * x;
    }

    return result;
}

//Recursive fast power
int recursivePower(int x, int n)
{
    if (n == 0)
        return 1;

    if (n % 2 == 0)
        return recursivePower(x * x, n / 2);
    else
        return x * recursivePower(x * x, (n - 1) / 2);
}

int main()
{
    int x, n;
    volatile int result1, result2;
    clock_t start, end;
    double time1, time2;

    printf("Enter base: ");
    scanf("%d", &x);
```

```

printf("Enter power: ");
scanf("%d", &n);

if (n < 0)
{
    printf("Enter a non-negative power.\n");
    return 0;
}

// Normal method timing
start = clock();

for (int i = 0; i < 1000000; i++)
{
    result1 = normalPower(x, n);
}

end = clock();

time1 = (double)(end - start) / CLOCKS_PER_SEC;

// Recursive method timing
start = clock();

for (int i = 0; i < 1000000; i++)
{
    result2 = recursivePower(x, n);
}

end = clock();

time2 = (double)(end - start) / CLOCKS_PER_SEC;

printf("\nNormal Method Result = %d\n", result1);

printf("Recursive Method Result = %d\n", result2);

printf("\nTime taken by Normal Method = %f seconds\n", time1);
printf("Time taken by Recursive Method = %f seconds\n", time2);

return 0;
}

```

5. Sample Output

```

Enter base: 2
Enter power: 20

Normal Method Result = 1048576
Recursive Method Result = 1048576

Time taken by Normal Method = 0.036088 seconds
Time taken by Recursive Method = 0.016925 seconds

```

6. Time and Space Complexity

Method	Time	Space
Normal (loop)	$O(n)$	$O(1)$
Recursive (fast power)	$O(\log n)$	$O(\log n)$

7. Summary

The recursive method halves the power every step, so it needs only about $\log n$ steps, while the normal method needs n multiplications. Hence the recursive method is faster for large n , as seen in the output. The recursive method uses $O(\log n)$ extra space for the function call stack, while the normal method uses $O(1)$.